

NOTE The `staticfiles` app has its own version of `runserver`, which is a development server that also hosts static content. If `staticfiles` is configured in your system, `runserver` will show in that grouping instead.

The rest of this section covers a few of the most commonly used management commands that aren't already discussed in detail in the appropriate part of the book. For example, the `test` management command got covered in chapter 8 along with other content on testing. For a full list of the built-in Django management commands, see <https://mng.bz/qOOK>.

9.1.2 *Changing a password*

Django assumes that if you have access to management commands, you have administrative powers. Nobody but your system administrators should be allowed in the server account that hosts your Django project. This makes sense: if they're in there, they can tamper with your `settings.py` file and turn off all the security checks anyhow.

If you are an admin, one of the things you might want to do is reset a user's password. The Django Admin provides a mechanism for this, but if you are locked out of the admin account, you need another way. The `changepassword` management command lets you change any user's password from the command line. It's pretty straightforward; run the command giving the username to reset, and then follow the prompts:

```
(venv) RiffMates$ python manage.py changepassword nicky
Changing password for user 'nicky'
Password:
Password (again):
Password changed successfully for user 'nicky'
```

In chapter 2, you used the `createsuperuser` management command to create an admin account. If you tried to use a password that wasn't particularly secure, Django warned you and gave you the option to bypass password validation. The `changepassword` command does no such thing: you are bound by the current password security level.

Password validation

If you've been snooping around in your `settings.py` file, you may have noticed the `AUTH_PASSWORD_VALIDATORS` configuration value. This list of dictionaries specifies which password validation code gets executed when a new password gets created. By default, there are four validators; they check for similarity to the username, minimum length, common passwords, and requiring a password to contain a number.

Like most configurable things in Django, you can write your own validator, which you'd then register in `AUTH_PASSWORD_VALIDATORS`. Details can be found in the documentation: <http://mng.bz/Xq6l>.